

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-xxxx-xxxx

**Rozšířená šablóna záverečnej práce na FEI STU
v Bratislave v systéme $\text{\LaTeX}$**

Bakalárska práca

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-xxxx-xxxx

**Rozšířená šablóna záverečnej práce na FEI STU
v Bratislave v systéme $\text{\LaTeX}$**

Bakalárska práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Ústav multimediálnych informačných a komunikačných technológií
Školiteľ:	Ing. Marek Vančo, PhD.

Podakovanie

Abstrakt

Kľúčové slová

záverečná práca, šablóna, L^AT_EX, formátovanie textu, citácie

Abstract

Keywords

Final thesis, template, L^AT_EX, text formatting, citations

Obsah

Úvod	7
1 Súčasný stav	9
1.1 Znalostné grafy	9
1.2 Klasické metódy extrakcie/extrakcia informácií z textu	10
1.3 Spracovanie dokumentov a layout analýza	11
1.4 Existujúce prístupy pre tvorbu znalostných grafov z textu	11
1.5 Medze	11
2 Návrh riešenia	12
2.1 Predspracovanie	12
2.1.1 Detekcia rozloženia	12
2.1.2 Extrakcia tabuliek	12
2.2 Trojkroková extrakcia	13
2.2.1 Detekcia v otvorenej doméne (ODD)	13
2.2.2 Úprava a spresnenie schémy (SR)	14
2.2.3 Schémou riadená extrakcia (SDE)	14
2.3 Vkladanie do databázy	14
3 Implementácia	16
3.1 Technologický stack	16
3.2 Kľúčové implementačné rozhodnutia	17
3.3 Prompt engineering pre slovenčinu	19
4 Experimenty a vyhodnotenie	21
4.1 Testovanie dáta a experimentálne prostredie	21
4.2 Priebeh extrakcie a kvantitatívne výsledky	21
4.3 Vizualizácia znalostného grafu	21
4.4 Porovnanie s baseline	21
4.5 Diskusia a limitácie	21
Záver	22
Literatúra	23
Použitie nástrojov umelej inteligencie	23

Zoznam značiek a skratiek

Úvod

v uvode obsiahnut problematiku

1 Súčasný stav

8 s. dokopy

1.1 Znalostné grafy

Znalostné grafy v poslednej dobe narástli na popularite v open-source komunite. Využívajú sa na ukladanie reálnych dát formulovaných ako entity a vzťahy. Hogan et al. definujú znalostný graf ako *graf údajov, ktorého uzly reprezentujú entity reálneho sveta a hrany reprezentujú vzťahy medzi týmito entitami* [1]. Generatívne veľké jazykové modely (LLM) trpia z veľkej miery halucináciou, nie sú schopné sa rozhodnúť alebo nemajú dostatočné znalosti pokiaľ nie sú natrénované v danej oblasti. V tom prichádza zmena s využitím znalostných grafov na využitie uchovania rozhodovacej logiky, vedomostí, zdrojového kódu a mnoho ďalších. Najznámejšími verejne dostupnými znalostnými grafmi sú DBpedia, Wikidata, YAGO.

V tejto práci modelujem údaje ako množinu trojíc (*triples*) v tvare (e_h, r, e_t) , kde e_h označuje začiatočnú entitu (*head*), e_t koncovú entitu (*tail*) a r vzťah medzi nimi. Takáto trojica zodpovedá orientovanej hrane $e_h \xrightarrow{r} e_t$ v orientovanom grafe, ktorý je podľa Hogan et al. [1] základným modelom pre znalostné grafy a je súčasne dátovým modelom štandardu RDF konzorcia W3C.

V projekte je hlavnou databázou na ukladanie dát grafová databáza s vlastnosťami (Property graphs), akou je napríklad Neo4j. Formálna definícia grafovej databázy s vlastnosťami podľa Hogan et al., ktorá uchováva dáta v grafe v podobe usporiadanej n-tice $G = (V, E, L, P, U, e, l, p)$. Platí: V je množina identifikátorov vrcholov (uzlov), E je množina identifikátorov hrán, L je množina typov, ktoré môžu niesť uzly aj hrany, P je množina mien vlastností, U je množina hodnôt vlastností, $e : E \rightarrow V \times V$ každú hranu priradí dvojici vrcholov (jej začiatočnému a koncovému uzlu), $l : V \cup E \rightarrow 2^L$ priraduje každému uzlu alebo hrane množinu typov, $p : V \cup E \rightarrow 2^{P \times U}$ priraduje každému uzlu alebo hrane množinu dvojíc (*vlastnosť, hodnota*). Intuitívne: vrcholy a hrany sú len identifikátory z univerzálnej množiny databázy, funkcia e špecifikuje štruktúru grafu (kto je s kým spojený), funkcie l a p nesú „sémantiku“, typy uzlov/hrán a ich atribúty.[1]

V porovnaní s relačnými databázami sa dá jednoduchšie pristupovať k viac skokovým údajom alebo cestám, kde sa vyhýbajú relačným joinom. V relačnej databáze by sme každý typ uchovávali v jednotlivých tabuľkách, zatiaľ čo v grafovej databáze je to oddelené typom entity alebo vzťahu. Taktiež relačné databázy sa nevedia dynamicky prispôbiť novým typom a preto si vyžadujú vopred definovanú schému. Remodelovanie takejto databázy je veľmi nákladne a časovo náročné. K dátam môžeme ľahko pristupovať

pomocou dopytovacieho jazyka Cypher, ktorý je súčasťou rodiny grafových dopytovacích jazykov spolu s SPARQL. Tieto jazyky podporujú okrem štandardných relačných operátorov aj operátory, ktoré rekurzívne vyhľadávajú cestu medzi entitmi ľubovoľnej dĺžky, čo je v tradičnom SQL náročne vyjadriteľné.

Alternatívou by mohla byť vektorová databáza, kde sa údaje ukladajú ako vektory v multidimenzionálnom priestore. Dáta sa dajú následne vyhľadávať pomocou sémantickej podobnosti a vrátia k najbližších susedov na základe kosínusovej vzdialenosti. Ich slabinou je strata explicitného relačného kontextu, ak otázka vyžaduje následovať reťaz $A \rightarrow B \rightarrow C$, čisto sémanticky podobné údaje môžu relevantnú väzbu minúť, pretože kľúčové údaje sú roztrúsené v dokumente [2]. Naopak znalostné grafy robia vzťahy explicitnými a umožňujú uvažovanie nad cestami, kde kľúčové uzly majú vzdialenosť n hrán (skokov).

Pri používaní LLM agentových riešení pri programovaní, agenti vedia z veľkej miery prečítať pri každej novej konverzácii nadbytočné množstvo súborov a tým rýchlo zaplniť kontextové okno konverzácie. Tento problém vieme neutralizovať využitím nástroja, akým je Graphify, kde kód projektu (funkcie, triedy, globálne premenné, ...) je štruktúrovaný v znalostnom grafe. Ďalším praktickým riešením využitím znalostných grafov je v prípade riešenia problémov opravy strojov. Súčiastky a stroje môžu predstavovať množinu uzlov a hrany vzťah medzi nimi. Ak sa naskytne nejaký problém so strojom, vieme daný problém označiť ako uzol a zoskupiť stroj a chybové súčiastky do komunity pomocou hrán. Pri prvom výskyte problému, opravár označí problém a kde bola chyba, a neskôr využitím LLM môžeme rýchlo identifikovať zdroj chyby pomocou znalostného grafu. Taktiež veľké využitie znalostných grafov nadobúda v oblasti kybernetiky a bezpečnosti (identifikácia hrozieb a útokov, forenzná analýza, detekcia anomálií, ...), vo vojenskej a obrannej infraštruktúre, medicíne, finančníctve a vo viacerých ďalších rôznych odvetviach.

Znalostné grafy dokážu odhalovať vzory, uzemňujú fakty pre hľadanie odpovedí, znížiť kontextové okno pri používaní LLM, znížiť halucinácie a cenu spracovania odpovede.

1.2 Klasické metódy extrakcie/extrakcia informácií z textu

1.5 s. Klasické NER/RE, prechod na LLM-based extraction, structured output, problém halucinácií a schéma-drift.

1.3 Spracovanie dokumentov a layout analýza

OCR a parsovanie PDF, problém tabuliek v plain-text extrakcii, DocLayout-YOLO ako riešenie detekcie štruktúrnych prvkov, vision LLM pre prevod obrázkov→štruktúrovaný výstup.

1.4 Existujúce prístupy pre tvorbu znalostných grafov z textu

3 s. najdôležitejšia sekcia

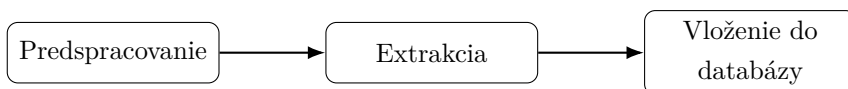
LangChain LLMGraphTransformer ako baseline ODKE+ pipeline (Open Domain Knowledge Extraction) — trojfázový návrh ODD → Schema Refinement → SDE GraphRAG (Microsoft) Špecifiká pre slovanské jazyky a právnu doménu (málo práce v tejto oblasti — tvoja medzera vo výskume)

1.5 Medze

Čo existujúce prístupy neriešia: slovenčina, právna doména, hybridné spracovanie text + tabuľky v jednej pipeline → motivácia pre vlastný návrh.

2 Návrh riešenia

Spracovanie transformuje PDF dokument Slovenského právneho odvetvia z neštruktúrovaného textu na štruktúrované dáta, ktoré sa uložia v znalostnom grafe (KG). Je to súvislá troj-kroková postupnosť: predspracovanie, extrakcia a vloženie do databázy. Tento tok dát je znázornený na obrázku č.1.



Obr. 1: Graf popisujúci spracovanie.

2.1 Predspracovanie

Čisté spracovanie PDF dokumentu nie je dostačujúce pre legislatívne dokumenty. Napríklad tabuľky, ktoré sú nositeľmi dôležitých, opisných a rozvíjajúcich vlastností, ako napríklad v zákone 222/2004 Z. z. [3] na strane 159, kde je kapitola/číselný znak spoločného colného sadzobníka a k nemu podliehajúci opis tovaru. Dokonca, sa táto tabuľka rozkladá na viacero strán. Pri čítaní tabuľky, ako reťazca by sa tieto informácie roztrúsili a zničil by sa tak kontext celej tabuľky. Vzorce sa taktiež vyskytujú naprieč dokumentom (str. 149) a v PDF dokumente su uchované ako obrázky - nástroj na čítanie textu z dokumentu by tento obrázok nerozpoznal a nenašiel.

2.1.1 Detekcia rozloženia

V tomto kroku sa každá strana dokumentu konvertuje na obrázok s rozlíšením 200 DPI. Aplikuje sa DocLayout-YOLO model, ktorý prejde a zdetekuje rozloženie každej strany. Tento detektor je nakonfigurovaný aby rozoznal päť cieľových tried: tabuľka, obrázok, diagram, izolovaný vzorec a popis vzorca. Detekcie s istotou pod 0,3 sú zahodené. Pre každú nájdenú detekciu, naväzujúci box je vystrihnutý z obrázku stránky a uložený ako bezstratový PNG súbor. Všetky susedné obrázky na základe strany (ak rozdiel strán je menší ako 2) sú zoskupené, aby tabuľky, ktoré sa rozliehajú na viacero strán, boli pri sebe ako jeden logický celok.

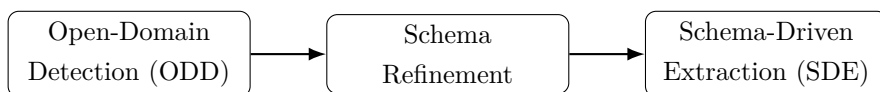
2.1.2 Extrakcia tabuliek

Každá skupina tabuliek je spracovaná v dvoch krokoch. Prvý krok, optický schopný Gemini model, prijme všetky orezané obrázky skupiny spoločne s system promptom, ktorý nariaďuje aby bol výstup tabuľka v podobe HTML. Tento prompt zakazuje sumarizáciu tabuľky a nariaďuje aby model zachoval doslovný text bunky. Druhý krok,

výsledný HTML výstup je poskytnutý ďalšiemu LLM volaniu, kde tento krát ma model za úlohu transformovať tabuľu do hierarchickej podoby vrcholov a vzťahov: tabuľka je označená ako koreňový vrchol, bunky každého riadku prvého stĺpca ako rodič, a potomok pre každú ďalšiu bunku daného riadku. Je to cesta ako zachovať význam tabuľky v znalostnom grafe. Vnútorne strany viac-stranovej tabuľky sú vyradené zo zoznamu strán PDF dokumentu, pre neskoršiu extrakciu aby sa predišlo duplicitám rovnakej informácie.

2.2 Trojkroková extrakcia

Po dokončení predspracovania, zvyšné strany dokumentu sú rozsekané na chunky textov a spracované cez trojkrovovú extrakciu inšpirovanú výskumom ODKE+ [4]. Táto extrakcia sa skladá z troch postupných krokov: detekcia v otvorenej doméne v a.j. „*open-domain detection*“ (ODD), úprava schémy v a.j. „*schema refinement*“ (SR) a v poslednom rade je to schémov riadená extrakcia, a.j. „*schema-driven extraction*“ (SDE). Tento tok je zobrazený na obrázku č. 2.



Obr. 2: Extrakcia inšpirovaná ODKE+ [4]

2.2.1 Detekcia v otvorenej doméne (ODD)

Cieľom tohto prvého kroku je nájsť, aké typy entít a vzťahov sa v dokumente vyskytujú. Text je rozdelený pomocou nástroja RecursiveCharacterTextSplitter, s veľkosťou chunku 1200 charakterov a prekryvom 200 charakterov. Relatívne väčšie okno textu je zvolené schválne. V tejto etape model GPT-5.4-mini je vynútený aby vyhľadal typy a nie konkrétne inštancie. Širšie kontextové okno napomáha aby model videl viac ako len jednu vetu včase, čo je veľmi dôležité v legislačných dokumentoch, kde napríklad slovo „*základ dane*“ je uvedené niekoľko viet predtým ako je použité.

Každý chunk c_i je paralelne poslaný LLM agentovi, ktorého system prompt inštruuje aby našiel a vrátil všeobecné ale rozlíšiteľné typy entít a typy vzťahov. Agentovi je explicitne napísané aby sa zameriaval na typy ako sú zákony, paragrafy, práva a povinnosti. Všetky diakritické znaky sú normalizované do ASCII aby znalostný graf používal jednotnú slovenskú výslovnosť bez diakritiky. Výstup tohto kroku je zjednotený zoznam schém chunkov: $S_{ODD} = \bigcup_{i=1}^{|C|} S_i$. Schému môžeme zobrazit ako $S_i = (T_V^{(i)}, T_E^{(i)})$, kde $T_V^{(i)}$ sú typy entít (vrcholov v znalostnom grafe) a $T_E^{(i)}$ sú typy vzťahov (hrán v znalostnom grafe) medzi entitami.

2.2.2 Úprava a spresnenie schémy (SR)

Zjednotené schémy z predošlého kroku detekcie (ODD), sú poslané na úpravu schémy modelu Gemini 2.5 Pro, spolu s existujúcou schémou znalostného grafu: $S^* = \Phi(S_{ODD}, S_O)$. Tento krok úpravy ma čistú linguistickú úlohu: rieši duplikáty spôsobené dikaritikou alebo vynechaním písmena (Dan a Daň alebo ClenskyStat a ClenskStat) a v neposlednom rade zjednocuje sémanticky podobné typy a synonymá (Doklad, DanovyDoklad a HodnoveryDoklad zjednotí do Doklad). Zároveň nedovoľuje aby sa spojili sémanticky odlišné typu ako Dokument a Osoba. System prompt vynucuje pre typy vzťahov aby boli písané pomocou UPPER_SNAKE_CASE (SCREAMING_SNAKE_CASE) a pre typy entít PascalCase. Taktiež sa uchováva merge log, ktorý mapuje všetky typy do akého typu entity alebo vzťahu boli zjednotené. Pomáha to pri debuggovaní alebo auditovaní.

2.2.3 Schémou riadená extrakcia (SDE)

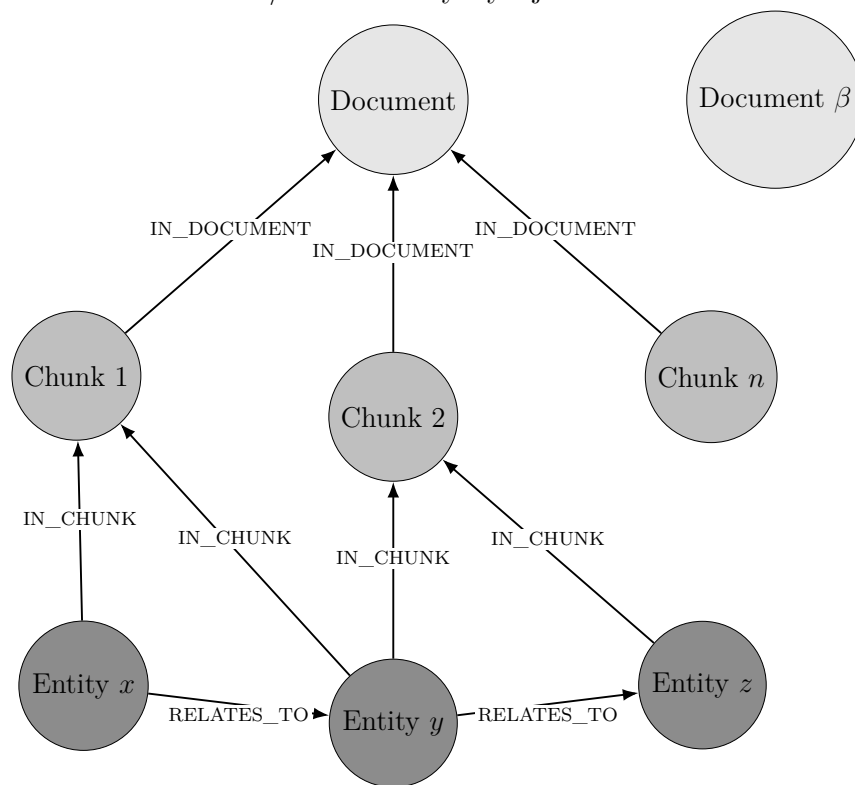
Posledný krok extrakcie je extrakcia pomocou natvrdo nanútenej schémy, v tomto prípade je to upravená S^* schéma. Text je v tomto kroku znovu rozdelený na menšie chunky s veľkosťou 1024 charakterov a prekryvom 128 charakterov. Tento menší chunk je podnietený odlišnou úlohou ako pri ODD. V tomto kroku GPT-5.4-mini model musí extrahovať konkrétne inštancie entít a vzťahov medzi nimi. Menšia veľkosť textu vynucuje model k extrahovaniu presnejších a lepšie uzemnených trojíc (e_h, r, e_t) , kde e_h je začiatočná entita vzťahu, e_t je koncová entita vzťahu a r je vzťah medzi danými entitami. Menšie kontextové okno znižuje pravdepodobnosť, že sa dva nesúvisiace fakty zmiešajú do jedného vzťahu. Chunky sú posielané paralelne s limitovanou súbežnosťou a pause-and-retry mechanizmom, v prípade vyčerpania užívateľových tokenov za minútu.

Pre každý chunk c_i , model vracia uzly a vzťahy, ktoré korešpondujú upravenej schéme S^* . Týmto krokom získame GraphDocument $G_i = (V_i, E_i, src_i)$. Následne ľahká kontrola a opracovanie, znormalizuje id uzlov do title case a pre vzťahy do UPPER_SNAKE_CASE (SCREAMING_SNAKE_CASE), a vynechá trojice ktorým chýba zdroj alebo cieľ. Extrahované uzly z chunku sú spojené s uzlom *Chunk* pomocou vzťahu IN_CHUNK. Neskôr vďaka tomuto vzťahu v kroku na získavanie dát, môžeme získať originálny text, ktorý podporuje dané trojice.

2.3 Vkladanie do databázy

Extrahované grafové dokumenty (GraphDocument) sú vo finálnom kroku vložené do Neo4j grafovej databázy. Uzly sú vytvorené pomocou MERGE nad ich normalizovaným ID (konkrétna inštancia entity napr. Colna Dan), tak že ich opakované zmienenia

medzi chunkami sú zjednotené do jednotnej inštancie entity. Vzťahy u taktiež vytvárané pomocou MERGE logiky aby sa predišlo duplicitným hranám. Narozdiel od pridanych extrahovaných uzlov a Chunk uzlov, je v štruktúre databázy pridaný uzol typu **Document**, ktorý nesie ID názvu dokumentu. V právnych dokumentoch sú to ID ako ZZ_2004_222_20260101, kde ZZ je zbierka zákona, 2004 rok, 222 číslo zákona a 20260101 dátum vygenerovania dokumentu. Všetky chunky z tohto dokumentu sú pospájané pomocou vzťahu **IN_DOCUMENT**. Výslednú štruktúru a hierarchiu databázy môžeme vidieť na obrázku 3. Výhodou tejto architektúry je zachovanie vzťahu pôvodu odkiaľ je daná entita prevzatá. Taktiež je veľmi jednoduché zistiť pri spracovaní viacero dokumentoch, v akom dokumente/zákone sa vyskytuje.



Obr. 3: Štruktúra znalostného grafu: dokument, chunk, entita and vzťah. Obrázok prevzatý z blogu Neo4j [5] a upravený na moje riešenie.

3 Implementácia

3.1 Technologický stack

Python

Python slúži ako orchestračný jazyk celého systému. Je zvolený pre svoju rozsiahlu podporu v oblasti LLM. Jeho dynamické typovanie a množstvo knižníc umožňujú rýchlu integráciu komponentov - od klasifikácie obrázkov cez veľké jazykové modely až po databázove ovládače. V kóde sa využíva aj jeho podpora typových anotácií a dátových tried.

LangChain

LangChain je knižnica navrhnutá na budovanie systémov okolo LLM. Poskytuje abstrakcie nad rôznymi poskytovateľmi jazykových modelov, čo znamená, že výmena jedného modelu za druhý si nevyžaduje prepísanie aplikačnej logiky. V projekte sa využíva najmä jeho schopnosť reťaziť operácie, spravovať kontextové okná, pracovať s nástrojmi a budovať agentické workflowy.

Neo4j

Neo4j je grafová databáza postavená na modeli grafov s vlastnosťami. Entity v grafe reprezentujú uzly s vlastnosťami, menovkami a vzťahy sú typové hrany. Tento typ služby som zvolil kvôli dobrému softvérovému riešeniu, kde sa dá spravovať databáza, vizualizovať dáta a narábať s nimi pomocou Cypher queries. Taktiež je tu veľmi dobré riešenie prepájania grafov s vektorovým úložiskom, kde rôzne uzly môžu byť vektorizované a prepojené pomocou ich unikátneho ID.

LLM modely

Projekt využíva viac poskytovateľov LLM modelov súčasne. Modely sú volené od ich schopnosti, či to je rýchlosť, cena, kvalita spracovania, reasoning a ich určenie. Sú to konkrétne modely GPT-5.4-mini od OpenAI a Gemini 2.5 Pro od Googlu. Modely sú ukázané v metrike cena vstup/výstup, tokenov za minútu a požiadaviek za minútu.

- GPT-5.4-mini: 0.75\$/4.50\$, 200 000 TPM a 500 RPM
- Gemini 2.5 Pro: 1.25\$/10.00\$, 2 000 000 TPM a 150 RPM

K dňu 25.4.2026

Správy a inštrukcie sa následne posielajú v dvoch vstupoch. Prvým je systémová správa (SystemMessage, system_prompt), ktorá definuje rolu, kontext a pravidlá,

ktorými sa model má riadiť. Druhou je ľudská správa (HumanMessage, user role), ktorá obsahuje samotný vstup používateľa.

DocLayout-YOLO

DocLayout-YOLO je špecializovaná neurónová sieť založená na architektúre YOLO, ktorá bola natrénovaná na úlohy detekcie prvkov v dokumentoch. Na rozdiel od všeobecných detektorov objektov, ktoré hľadajú autá alebo ľudí, tento model rozpoznáva dokumentové prvky ako sú odseky textu, nadpisy, tabuľky, obrázky, vzorce, popisy obrázkov a ďalšie. Výsledkom je štruktúrovaná reprezentácia vizuálneho rozloženia stránky s orezanými obdĺžnikmi a ich pravdepodobnosťami. Na to aby sa tento nástroj dal použiť potrebujeme konverziu dokumentu v PDF formáte na obrázky. To zaručí knižnica pdf2image, ktorá premení PDF dokument na rastrové obrázky.

3.2 Kľúčové implementačné rozhodnutia

Mechanizmus semaforu

Pri spracovávaní dokumentu pomocou LLM modelu, ktoré sú rozdelené na desiatky či i stovky chunkov, nastávajú 2 problémy. Prvým problémom je, keď posielame veľa požiadaviek paralelne, tak prťažíme API a vyčerpáme limit tokenov za minútu (TPM - Tokens Per Minute). Druhým je, keď jedna požiadavka zlyhá kvôli týmto limitám, zvyšné požiadavky s istotou zlyhajú taktiež. Tieto problémy riešim pomocou koordinačného mechanizmu.

Inicializácia začína vytvorením 3 kontrolných prvkov. Prvým je semafor, ktorý riadi konkurenčný limit, koľko úloh naraz môže byť obsluhovaných. V tomto implementovaní je to 5 konkurenčných úloh. Zabraňuje to systému aby neodoslal stovky požiadaviek naraz. Druhým prvkom je globálny prepínač *pause switch*, ktorý začína v zapnutej pozícii. Pokiaľ tento prepínač je v zapnutej pozícii, systém môže voľne posilať API požiadavky. Akonáhle prepínač sa dostane do pozície vypnutej, každá pracovná úloha v systéme sa zastaví na rovnakom synchronizačnom bode a čaká. Posledným prvkom je zámok, ktorý povoľuje v prípade chyby alebo zlyhania pracovať iba na jednej úlohe.

Po nastavení týchto riadiacich prvkov, je každému chunku pridelená asynchrónna úloha. Každá úloha sa riadi rovnakou rutinou a má povolené až tri pokusy na dokončenie svojho procesu. Na začiatku každého pokusu úloha naprv overí stav globálneho prepínača. Ak je prepínač vypnutý, úloha čaká, kým sa znova nezapne.

Keď je prepínač aktívny, úloha sa pokúsi získať miesto v semafore. Akonáhle miesto získa, spustí samotnú extrakcie alebo detekciu volaním LLM modelu. Ak požiadavka prebehne úspešne, úloha skončí a vráti svoj výsledok. Ak volanie zlyhá vyhodnotením

výnimky a úloha má ešte k dispozícii ďalšie pokusy, nastáva situácia, kedy sa úloha pokúsi získať zámok. Ten zaručuje, že iba úplne prvá úloha, ktorá zlyhanie zaregistruje, sa stáva zodpovednou za jeho riešenie. Táto úloha následne vypne globálny prepínač, čím okamžite pozastaví všetky ostatné úlohy v systéme. Následne sa systém uspí na 60 sekúnd, čo je dostatočne dlhý čas na to, aby sa rate limit modelu obnovil, a následne sa prepínač opäť zapne.

Ak pracovná úloha nakoniec vyčerpá všetky tri pokusy bez úspechu, vzdá sa a propaguje výnimku ďalej, namiesto toho aby blokovala zvyšok systému. Medzitým boli všetky úlohy spustené v rovnakom okamihu, skrz mechanizmus task creation a počas celého procesu bežia paralelne v rámci jedného event loopu. Hlavná rutina čaká pomocou agregáčnej operácie gather, kým každá jednotlivá úloha neskončí.

Precíznosť tohto návrhu spočíva v spôsobe, akým zaobchádza so zlyhaním ako so zdieľaným signálom. Jediné zlyhanie je interpretované ako dôkaz toho, že celý spracovateľský systém je obmedzený, a preto je cooldown aplikovaný globálne a nie individuálne. Zámok, tu zohráva zásadnú rolu, pretože bez neho by mohli desiatky úloh detekovať to isté zlyhanie v rovnakom okamihu a spustiť svoj 60 sekundový interval nezávisle.

Štruktúrovaný výstup

V kontexte spracovania prirodzeného jazyka prostredníctvom LLM predstavuje štruktúrovaný výstup (a. j. structured output) zásadnú techniku, ktorá zaručuje, že odpoveď volania modelu bude zodpovedať definovanej schéme. Vďaka tomu sa eliminuje chybovosť, ktorú zapríčiňuje parsovanie výstupných dát z modelu, zaručí plynulosť spracovania a zdieľania dát vo workflowe. Dáta môžeme jednoducho deserializovať do Python objektov (dictionary, dataclass alebo Pydantic modelu). V prostredí knižnice LangChain existujú dva odlišné spôsoby, ktoré majú svoje špecifické vlastnosti, výhody a obmedzenia v závislosti od použitého modelu.

Prvý prístup, ktorý sa v kóde uplatňuje predovšetkým na modely od spoločnosti OpenAI, je stratégia poskytovateľa (ProviderStrategy). Tento prístup využívajú modely, ktoré natívne podporujú štruktúrovaný výstup (OpenAI, Anthropic, xAI, Gemini)[6]. Je to principiálny prístup pri komunikácii s modelom skrz agentovu vrstvu. Schéma očakavaného výstupu sa pri tomto prístupe odovzdáva ako parameter formátu odpovede (response_format) priamo pri vytváraní agenta.

Hlavnou výhodou tejto stratégie je deterministická záruka, že vrátený objekt bude validný voči zadanej schéme. Ďalším významným prínosom je integrácia s agentovým ekosystémom, čo umožňuje rozšírenie o nástroje, kontextovú pamäť a iteratívne uvažo-

vanie bez nutnosti meniť spôsob, akým sa štruktúrovaný výstup spracováva. V kóde projektu sa tento prístup uplatňuje pri kľúčových úlohach, akými sú detekcia v otvorenej doméne a schémou riadená extrakcia.

Druhý prístup, použitý v projekte pri komunikácii s modelmy Gemini, je metóda priameho štruktúrovaného výstupu (`with_structured_output`). V kóde sa tento princíp kombinuje s metódou založenou na prevode schémy do formátu JSON Schema. Tento prístup pracuje na úrovni samotného modelu bez agentovej vrstvy. Metóda vracia nový Runnable, ktorý pri každom volaní vloží schému do natívneho parametra API (`response_format` pre OpenAI a pre Gemini `response_mime_type` alebo `response_json_schema`) a automaticky parsuje odpoveď[7, 8]. Ak je schéma Pydentic trieda, tak požiadavka vráti inštanciu tejto triedy, a pri JSON Schema zas vráti dict.

Výhodou tohto prístupu je jeho jednoduchosť volania a priamy návrat výsledku bez potreby extrahovania z vnoreného kontextového výstupu. V kóde sa tento prístup uplatňuje pri spracovávaní tabuliek do HTML, transformácií HTML tabuliek do grafovej reprezentácie a pri úprave schémy.

3.3 Prompt engineering pre slovenčinu

Dôraz na ASCII v názvoch typov (ako constraint pre property graph labels), chain-of-thought pokyny, few-shot príklady, riešenie skloňovania.

Kvalita spracovania zadania modelu závisí nielen od schopností zvoleného modelu, ale aj od formy, v akej je modelu zadaná inštrukcia. Prompty sa musia zamerať na tieto triedy:

- Štruktúra promptu - modely reagujú výrazne presnejšie ak je vstup členený do sekcií. Prompt je písaný v Markdown formáte, kde nadpisy a podnápisy sú označené pomocou # alebo ##. Typické sekcie promptu bývajú: ÚLOHA, KONTEXT, VSTUP, POKYNY a PRÍKLADY.
- Konkrétnosť inštrukcie - inštrukcie musia byť presné a jazykovo presne definované. Ak model nemá explicitne napísané čo a ako má spraviť, výstup úlohy nemusí zodpovedať našim požiadavkam.
- Pravidlá a príklady - model musí mať presne ohraničené čo môže. Ak model explicitne neohraničíme pravidlami, model nemusí dodržiavať inštrukcie. Taktiež pár príkladov ako ma úlohu vykonať dopomôže kvalite spracovania (few-shot learning).

- Špecifiká pre slovenčinu - modelom je vynútené ako dany typ vzťahu alebo entity majú spracovať. Entity alebo uzly v grafe sú označované ako Title Case, typy entít PascalCase a vzťahy UPPER_SNAKE_CASE (SCREAMING_SNAKE_CASE). Dané triedy musia byť spracované bez diakritiky. Keby nie sú normalizované na nediakritické slová mohlo by sa stať že by daný extrahovaný typ by bol s diakritikou, v neskoršom spracovaní ďalšieho chunku bez diakritiky, a vznikli by duplicitné údaje.

Príklad systémovej inštrukcie na úpravu schémy pre model Gemini 2.5 Pro:

```
# ROLA
Si expertný ontológ a dátový inžinier špecializujúci sa na znalostné grafy.
Tvojou úlohou je vyčistiť, normalizovať a deduplikovať schému entít a vzťahov.

# CIELE
1. **Deduplikácia a sémantické zjednotenie**: Zlúč synonymá a sémanticky blízke typy do jedného kanonického typu.
2. **Normalizácia formátu**:
  - Uzly: PascalCase (napr. DanovePriznanie).
  - Vzťahy: UPPER_SNAKE_CASE (napr. MA_POVINNOST).
  - **STRIKTNÉ PRAVIDLO**: Odstráň všetku diakritiku (á->a, č->c, atď.) zo všetkých názvov.
3. **Ontologická integrita (KRITICKÉ)**:
  - NIKDY nezlučuj entity z rôznych kategórií: Aktér (Osoba) vs. Proces/Dokument (Ziadost), Miesto (Kraj) vs. Politický útvar (Stat).
  - NIKDY nezlučuj protichodné vzťahy (VSTUPUJE_DO vs. VYSTUPUJE_Z).
4. **Generalizácia**: Príliš špecifické uzly nahraď všeobecnejšími (napr. "Sluha" -> "Osoba"), ak to neznižuje zrozumiteľnosť právneho/biznisového kontextu.

# PRAVIDLÁ PRE ZÁKLADNÚ ONTOLÓGIU
- Ak je poskytnutá Základná ontológia, má prednosť v pomenovaní.
- Ak surový typ (Open Domain) zodpovedá typu v základnej ontológii, mapuj ho naň.
- Ak je surový typ unikátny a dôležitý, pridaj ho ako nový typ.
```

4 Experimenty a vyhodnotenie

9 s.

4.1 Testovanie dáta a experimentálne prostredie

1 s. Opis vybraných právnych textov (konkrétny zákon alebo colný sadzobník), štatistika vstupu (strany, tabuľky, odhadované entity), hardvér, použité verzie modelov.

4.2 Priebeh extrakcie a kvantitatívne výsledky

3 s.

Metriky z tvojho pipeline:

- Počet chunkov po oboch chunkovaniach - ODD: počet detegovaných typov uzlov a relácií per chunk, histogram - Refinement: kompresia schémy (napr. 150 surových typov → 45 kanonických), merge log s ukážkami ("Dan" + "Daň" + "dan" → "Dan") - SDE: počet extrahovaných inštancií uzlov a relácií - Tabuľková vetva: počet detegovaných tabuliek vs. manuálne spočítané, úspešnosť prevodu - Čas behu jednotlivých fáz, počet LLM volaní, odhad nákladov

4.3 Vizualizácia znalostného grafu

1.5 s. Ukážka subgrafu z Neo4j Browser (napr. PravnyPredpis → Paragraf → Polozka refazec), porovnanie vstupu (PDF stránka) s výstupom (zodpovedajúci subgraf).

4.4 Porovnanie s baseline

1 s. Napr. LLMGraphTransformer bez schema refinement kroku — ukáže pridanú hodnotu tvojho trojfázového návrhu. Alebo porovnanie s ODD-only (bez refinementu).

4.5 Diskusia a limitácie

zaver 1 s. Čo sa ukázalo ako silné stránky (konsolidácia schémy, hybridné spracovanie tabuliek), kde sú slabé miesta (náklady na LLM, závislosť na kvalite PDF, viacstránkové tabuľky s rozdielnou štruktúrou).

Záver

Literatúra

1. A. HOGAN E. Blomqvist, M. C. et al. *Knowledge Graphs* [online]. arXiv, 2021 [cit. 2026-04-26]. Dostupné z eprint: 2003.02320.
2. MAYO, M. *Vector Databases vs. Graph RAG for Agent Memory: When to Use Which.* [online]. MachineLearningMastery, 2026 [cit. 2026-04-26]. Dostupné z: <https://machinelearningmastery.com/vector-databases-vs-graph-rag-for-agent-memory-when-to-use-which/>.
3. MINISTRY OF JUSTICE OF THE SLOVAK REPUBLIC. *Slov-Lex: Legal and Information Portal* [online]. Ministerstvo spravodlivosti Slovenskej republiky [cit. 2026-04-20]. Dostupné z: <https://www.slov-lex.sk/>.
4. KHORSHIDI, S. et al. *ODKE+: Ontology-guided open-domain knowledge extraction with LLMs* [online]. arXiv, Apple Inc., 2025 [cit. 2026-04-20]. Dostupné z eprint: 2509.04696.
5. KOHLWEY, E. *GraphRAG field guide: Navigating the world of advanced RAG patterns* [online]. Neo4j Developer Blog, 2024 [cit. 2026-04-20]. Dostupné z: <https://neo4j.com/blog/developer/graphrag-field-guide-rag-patterns/>.
6. LANGCHAIN, INC. *Structured output, LangChain documentation* [online]. LangChain, Inc. [cit. 2026-04-22]. Dostupné z: <https://docs.langchain.com/oss/python/langchain/structured-output>.
7. *ChatGoogleGenerativeAI reference* [online]. [cit. 2026-04-22]. Dostupné z: https://reference.langchain.com/python/langchain-google-genai/chat_models/ChatGoogleGenerativeAI/response_schema.
8. *LangChain OpenAI reference* [online]. [cit. 2026-04-22]. Dostupné z: https://reference.langchain.com/python/langchain-openai/chat_models/base/BaseChatOpenAI/with_structured_output.

Použitie nástrojov umelej inteligencie